

Image Analyzer

File 1 – Open Image

- Imported the PIL library.
- Stored the image location in `image_path`.
- Opened the image using `Image.open()`.
- Displayed the image using `image.show()`.
- Printed a success message after opening the image.

This file is the starting point of the project. Before doing any processing, the image must be loaded into Python and displayed to check whether it is read correctly.

File 2 – Read Image Details (`read_image.py`)

- Opened an image.
- Printed the image width.
- Printed the image height.
- Printed the image size.
- Printed the image format.
- Printed the image color mode.

This helps to understand the basic properties of an image. These details are useful before performing image processing operations like resizing or analysis.

File 3 – Resize Image (`resize_image.py`)

- Opened the image.
- Resized it to **500 × 500 pixels**.
- Saved the resized image in the images folder.
- Displayed the resized image.
- Printed a confirmation message.

Resizing changes the dimensions of the image without changing its content. It is commonly used to prepare images for machine learning models or reduce storage space.

File 4 – Functions (`4functions.py`)

- Created `open_image()` to open and return an image.
- Created `show_image()` to display the image.

- Created `print_details()` to display width, height, format, and mode.
- Added a `save_image()` function to save the processed image.
- Used `Path()` to create the output folder if it does not exist.
- Called the functions one by one in the main program.

Instead of writing all the code in one place, I divided it into functions. This makes the program easier to read, reuse, and modify.

File 5 – Loops (loops.py)

- Created a list containing three image paths.
- Used a for loop to open each image.
- Printed the width, height, and format of every image.

A loop allows the same code to work for multiple images automatically instead of writing separate code for each image.

File 6 – OOP (oops.py)

- Created an `ImageAnalyzer` class.
- Used a constructor to open the image when an object is created.
- Declared `__path` and `__image` as private variables.
- Created `get_path()` to access the image path.
- Created `set_path()` to change the image.
- Created `details()` to print image information.
- Created `show()` to display the image.
- Created `resize()` to resize and save the image.
- Created an object and tested all the methods.

This file was created to understand Object-Oriented Programming and Encapsulation. Keeping image data inside a class makes the program more organized and secure.

File 7 – Exception Handling (exception.py)

- Allowed the user to enter an image name, file path, or folder path.
- Automatically selected a default image when the user pressed Enter.
- Displayed all available images before asking for input.
- Used try, except, and finally.
- Handled:

- File Not Found
 - Unsupported or Corrupted Image
 - Permission Error
 - Unexpected Errors
- Printed image details if the image loaded successfully.

Users may enter incorrect paths or unsupported files. This file prevents the program from crashing and displays meaningful error messages.

File 8 – Image Combiner (image.png creator)

- Read all image files from the current folder.
- Converted every image to PNG (RGBA) format.
- Resized all images to the same size.
- Combined all images into a single image.
- Saved the final combined image as image.png.
- Displayed the combined image.

This file was created to combine multiple images into one image. It also ensures all images have the same size before combining them.